高性能 TCP 网关 (MVP 2) 三周开发方案

> **版本规范**：Route C V23.9 Absolute Final Sealed Edition
> **核心战略**：MVP First, Pro Later
> **执行原则**：明确边界与工程约束，防 OOM / DoS / UAF
> **工作流**：周一至周六手敲推进代码并同步更新 `main` 与 `review/weekX/dayY` 分支，周日统一复盘理解。

🟢 第一周：协议解析与基础联通 (核心闭环)

目标：完成基于 Epoll LT 的网络收发与基础 HTTP 解析。

- **Day 0 (前期准备)**
  - `chore: init project structure for WebServer` (初始化项目结构与 CMakeLists)
- **Day 1 (套接字建立)**
  - `feat: implement socket bind and listen` (创建监听 Socket 并绑定端口)
  - `feat: add detailed comments to socket init code` (补充详细注释)
- **Day 2 (非阻塞与接收)**
  - `feat: implement accept loop and non-blocking socket` (实现非阻塞 Accept 循环)
- **Day 3 (多路复用)**
  - `feat: refactor server to use Epoll event loop` (重构服务端引入 Epoll LT 事件循环)
- **Day 4 (应用层缓冲与边界)**
  - `feat: implement dynamic buffer and handle TCP sticking/splitting` (实现动态缓冲区，处理 TCP 粘包与 `\r\n\r\n` 请求头切割)
- **Day 5 (本周四)：完整的 HTTP 协议解析与边界防御**
  - 解析 Request Line（Method, URI, Version）。
  - 解析 Headers（提取 `Content-Length`）。
  - **DoS 防御 1**：发现 `Chunked` 直接返回 `501 + close`。
  - **DoS 防御 2**：在寻找 `\r\n\r\n` 过程中，只要 `inbuf.size() > 8192` 立即触发 `431 + close`。
- **Day 6 (本周五)：基础路由与响应写回**
  - 针对合法的 GET/POST 请求，构造 `200 OK` 响应（暂用静态字符串）。
  - 实现 best-effort 写回：写不完或遇到 EAGAIN 时，将剩余数据存入 `outbuf`，并注册 `EPOLLOUT` 事件等待下次可写。
- **Day 7 (本周日)：代码复盘与机制理解**
  - **复习重点**：结合 VSCode 调试，走通 Epoll LT 模式下可读/可写事件的触发时机；深刻理解非阻塞 I/O 必须配合应用层 Buffer（`inbuf`/`outbuf`）的核心原因。

🟢 第二周：架构约束与资源管理 (“把系统关进笼子”)

目标：实现单 Reactor 单 EventLoop 下的全局串行约束，确保系统在极端压力下不死锁、不爆内存。

- **Day 1：管线化与 ReadyQueue**
  - **防假死**：设置单连接每轮最多处理 5 个完整请求。若超限但 Buffer 仍有完整请求，将该连接加入 `ReadyQueue`，下一轮继续处理（严禁依赖下一次 EPOLLIN）。
- **Day 2：读写预算控制**

    - 设置单轮读取预算 `kMaxReadBytesPerEvent = 256KB`。
    - 设置单轮写入预算 `kMaxWriteBytesPerEvent = 256KB`。
- **Day 3：连接生命周期与防 UAF**
    - 实现**唯一的**关闭入口函数，置 `closing=true`。
    - **defer 释放流程**：`closing=true` -> 从 ReadyQueue 摘除 -> `epoll_ctl(DEL)` -> `close(fd)` -> 清理 fd 映射 -> 加入 `pending_close_queue` 在本轮末尾统一释放。
- **Day 4：全局内存背压防御 (Inflight Budget)**
    - 引入 `global_inflight_bytes` 计数器（默认 512MB）。追加 Buffer 前执行 `budget_check`，超限返回 `503 + close`。
    - **Body 限制**：`Content-Length > 8MB` 立即返回 `413 + close`。
- **Day 5：IP 级并发限流 (Token Bucket)**
    - 实现 `ip_token_bucket`：基于 LRU（默认最大条目 100k），TTL 10分钟。
    - 容量 200，每秒补充 50 token；耗尽则对新连接返回 `429 + close`。
- **Day 6：连接数封顶与拒绝逻辑**
    - 实现全局 `max_conns = 200k` 限制。
    - 超限时走 Day 3 的唯一关闭流程并递增 `conn_reject_total` 计数。
- **Day 7：代码复盘与机制理解**
    - **复习重点**：深挖为什么必须用 `pending_close_queue` 延迟释放（防并发回调 UAF）；检查所有计数器是否严格遵循“在哪加，就在哪减”的守恒定律。

🟢 第三周：超时控制、观测性与压测交付

目标：完善超时断连，输出标准 Metrics，跑通压测并落盘证据。

- **Day 1：定时器基础与超时打底**
    - 实现或引入时间轮/最小堆定时器结构。
    - 挂载 `header_timeout` (默认 10s) 和 `idle_keepalive_timeout` (默认 60s)。
- **Day 2：Slowloris 防御机制**
    - 动态计算 `body_deadline = now + min(120s, max(30s, Content-Length / 65536))`。
    - 实现 `body_timeout` (120s) 上限截断。
- **Day 3：Metrics 统一打点监控**
    - 接入 `std::atomic` 计数器。
    - 每秒输出一次标准日志到 `metrics.log`，包含 `uptime_s`，`rss_kb`，`accept_total`，`requests_total`，`reject_total`，`global_inflight_bytes_current` 等。
- **Day 4：Sanitizer 内存与并发自测**
    - 使用 `-O1 -g -fsanitize=address,undefined` (asan_ubsan) 编译并跑核心压力用例 ≥ 5分钟。
    - 修复潜在的内存越界或泄漏告警。
- **Day 5：标准化压测验证**
    - 使用脚本 `wrk -t8 -c100 -d35s --latency` 跑验收，扣除前 5s warmup。
    - 验证复用率指标：`requests_total / accept_count > 10`。
- **Day 6：DoD 验收与证据落盘**
    - 跑 3 轮压测取中位数，保留 raw latency samples。
    - 完成目录 `results/webserver/yyyymmdd_machine/` 下的 `machine.txt`，`build_flags.txt`，`bench_cmd.txt`，`summary.md` 证据落盘。
- **Day 7：封板复盘与面试对齐**

- **复习重点**：梳理项目的 Evolution Story。能够流畅复述：怎么发现管线化饥饿引入 ReadyQueue 的；怎么发现大包攻击并加 8MB 限制的。